

Image Processing and Automation Pipeline Using Lambda & S3 Events

1. Create S3 Buckets- Create two buckets

The screenshot shows the AWS Management Console 'Create bucket' page. The 'General configuration' section is active, showing the 'AWS Region' as 'Europe (Stockholm) eu-north-1'. Under 'Bucket type', 'General purpose' is selected. The 'Bucket name' is 'image-source-sakshi'. Below this, there's a section for 'Copy settings from existing bucket - optional' with a 'Choose bucket' button. The 'Object Ownership' section is also visible.

This screenshot shows the 'Create bucket' page for the second bucket, 'image-processed-sakshi'. The configuration is similar to the first bucket, with 'General purpose' selected as the bucket type and the name 'image-processed-sakshi' entered.

This screenshot shows the 'General purpose buckets' list in the AWS Management Console. It displays two buckets: 'image-processed-sakshi' and 'image-source-sakshi', both in the 'Europe (Stockholm) eu-north-1' region, created on December 10, 2025.

	Name	AWS Region	Creation date
☐	image-processed-sakshi	Europe (Stockholm) eu-north-1	December 10, 2025, 12:15:58 (UTC+05:30)
☐	image-source-sakshi	Europe (Stockholm) eu-north-1	December 10, 2025, 12:15:19 (UTC+05:30)

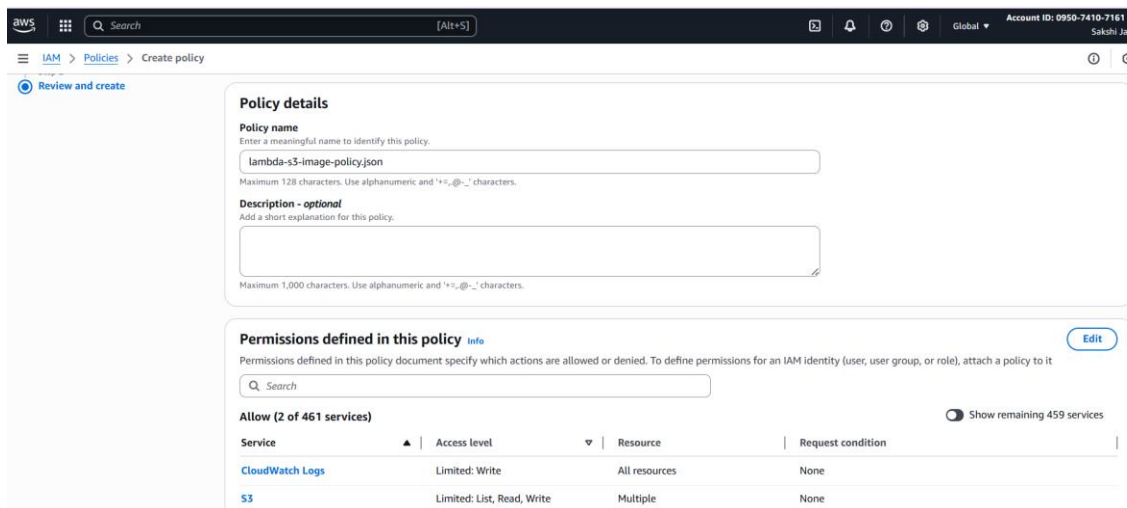
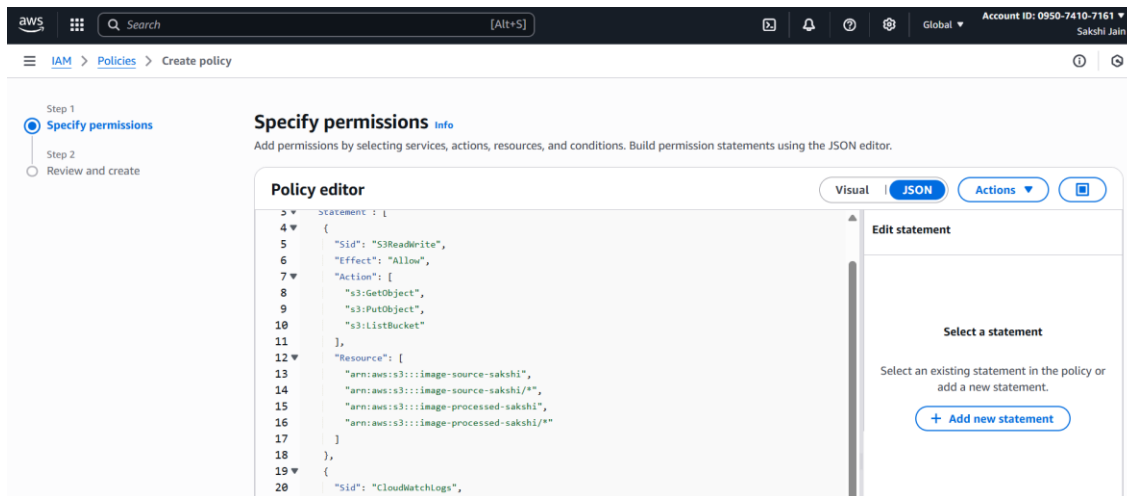
2. Create IAM Role for Lambda

Lambda requires permissions to read from source bucket, write to destination bucket, and write CloudWatch logs.

Create IAM policy

```
{  
  "Version": "2012-10-17",
```

```
"Statement": [  
  {  
    "Sid": "S3ReadWrite",  
    "Effect": "Allow",  
    "Action": [  
      "s3:GetObject",  
      "s3:PutObject",  
      "s3:ListBucket"  
    ],  
    "Resource": [  
      "arn:aws:s3:::image-source-sakshi",  
      "arn:aws:s3:::image-source-sakshi/*",  
      "arn:aws:s3:::image-processed-sakshi",  
      "arn:aws:s3:::image-processed-sakshi/*"  
    ]  
  },  
  {  
    "Sid": "CloudWatchLogs",  
    "Effect": "Allow",  
    "Action": [  
      "logs:CreateLogGroup",  
      "logs:CreateLogStream",  
      "logs:PutLogEvents"  
    ],  
    "Resource": "*"   
  }  
]  
}
```



IAM → Role (your Lambda role) → Permissions → Edit policy → JSON

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"

```

```
    ],  
    "Resource": "*"    
  },  
  {  
    "Effect": "Allow",  
    "Action": [  
      "s3:GetObject",  
      "s3:PutObject",  
      "s3:ListBucket"  
    ],  
    "Resource": [  
      "arn:aws:s3:::image-source-sakshi",  
      "arn:aws:s3:::image-source-sakshi/*",  
      "arn:aws:s3:::image-processed-sakshi",  
      "arn:aws:s3:::image-processed-sakshi/*"  
    ]  
  }  
]  
}
```

Create role and attach policy

aws

Search

[Alt+S]

Global

Account ID: 0950-7410-7161

Sakshi Jain

IAM

Roles

Create role

Step 1

Step 2

Step 3

Select trusted entity

Add permissions

Name, review, and create

Select trusted entity

Trusted entity type

Use case

Trusted entity type

☒ AWS service

☐ AWS account

☐ Web identity

☐ SAML 2.0 federation

☐ Custom trust policy

Allow AWS services like EC2, Lambda, or others to perform actions in this account.

Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

Create a custom trust policy to enable others to perform actions in this account.

Use case

Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

Lambda

Choose a use case for the specified service.

Use case

Lambda

CloudShell

Feedback

Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Permissions policies (2)

Info

Simulate

Remove

Add permissions

You can attach up to 10 managed policies.

Filter by Type

Search

All types

< 1 >

☐	Policy name	Type	Attached entities
☐	lambda-s3-image-policy	Customer inline	0
☐	lambda-s3-image-policy.json	Customer managed	1

aws

Search

[Alt+S]

Global

Account ID: 0950-7410-7161

Sakshi Jain

IAM

Roles

Create role

Step 1

Step 2

Step 3

Select trusted entity

Add permissions

Name, review, and create

Name, review, and create

Role details

Step 1: Select trusted entities

Role details

Role name

lambda-s3-image-role

Description

Allows Lambda functions to call AWS services on your behalf.

Step 1: Select trusted entities

Trust policy

1

2

3

4

5

6

{

"Version": "2012-10-17",

"Statement": [

{

"Effect": "Allow",

"Action": [

Lambda > Functions > Create function

Create function

info

Choose one of the following options to create your function.

☒ Author from scratch

Start with a simple Hello World example.

☐ Use a blueprint

Build a Lambda application from sample code and configuration presets for common use cases.

☐ Container image

Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

imageProcessor

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime

info

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.12

Architecture

info

Choose the instruction set architecture you want for your function code.

☐ arm64

☒ x86_64

Permissions

info

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ Change default execution role

Execution role

Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☐ Create a new role with basic Lambda permissions

☒ Use an existing role

☐ Create a new role from AWS policy templates

Existing role

Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

lambda-s3-image-role

[View the lambda-s3-image-role role](#) on the IAM console.

► Additional configurations

Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

Cancel

Create function

Lambda → Functions → imageProcessor
Go to the Configuration tab.

aws

Search

[Alt+S]

Europe (Stockholm)

Account ID: 0950-7410-7161

Sakshi Jain

Lambda > Functions > imageProcessor > Edit basic settings

Description - optional

Memory

info

Your function is allocated CPU proportional to the memory configured.

128

MB

Set memory to between 128 MB and 10240 MB.

Ephemeral storage

info

You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#).

512

MB

Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart

info

Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#). For Python and .NET runtimes, [view pricing](#).

None

Supported runtimes: .NET 8 (C#/.NET), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14.

Timeout

1

min

0

sec

Execution role

Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☒ Use an existing role

☐ Create a new role from AWS policy templates

Info

Tutorials

>

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

^

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

[Learn more](#)

Start tutorial

Still on the Configuration tab.
Click Environment variables (left side) → Edit.

[Lambda](#) > [Functions](#) > [imageProcessor](#) > Edit environment variables

Edit environment variables

Environment variables
You can define environment variables as key-value pairs that are accessible from your function code. These are useful to store configuration settings without the need to change function code. [Learn more](#)

Key	Value	
OUTPUT_BUCKET	image-processed-sakshi	Remove
THUMBNAIL_SIZE	128	Remove
CONVERT_TO_WEBP	true	Remove

[Add environment variable](#)

▼ Encryption configuration

Encryption in transit [Info](#)
☐ Enable helpers for encryption in transit

AWS KMS key to encrypt at rest
Choose an AWS KMS key to encrypt the environment variables at rest, or simply let Lambda manage the encryption.

☒ (default) aws/lambda

☐ Use a customer master key

Create your own Pillow Lambda Layer (no Docker)

Open CloudShell

Build the layer contents

1. Create a working folder

mkdir pillow-layer

cd pillow-layer

2. Create the "python" folder required by Lambda layers

mkdir python

3. Install Pillow into that folder

pip3 install "Pillow==11.0.0" -t python/

4. Zip it

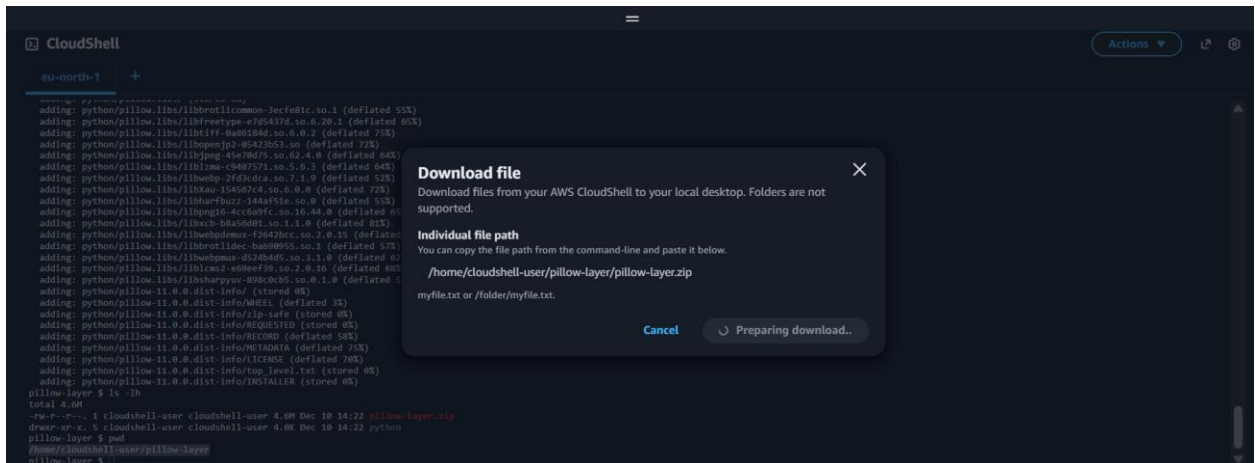
zip -r pillow-layer.zip python

CloudShell

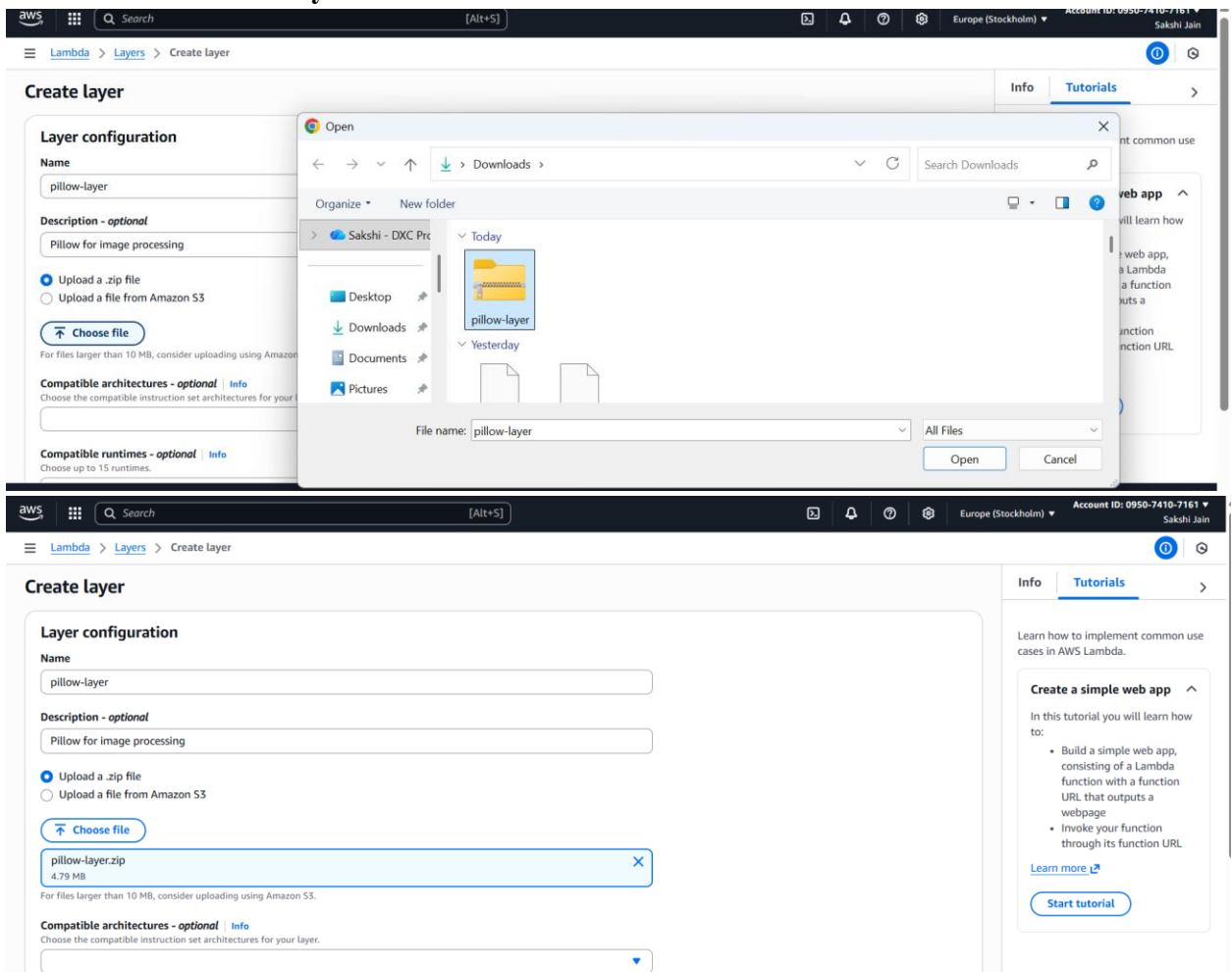
eu-north-1 +

```

- $ mkdir pillow-layer
- $ cd pillow-layer
pillow-layer $ mkdir python
pillow-layer $ pip3 install "Pillow==11.0.0" -t python/
Collecting Pillow==11.0.0
  Downloading pillow-11.0.0-cp39-cp39-manylinux_2_28_x86_64.whl (4.4 MB)
    |#####| 4.4 MB 18.9 MB/s
Installing collected packages: Pillow
Successfully installed Pillow-11.0.0
pillow-layer $ zip -r pillow-layer.zip python
adding: python/ (stored 0%)
adding: python/PIL/ (stored 0%)
adding: python/PIL/_imagingplugin.py (deflated 69%)
adding: python/PIL/features.py (deflated 75%)
adding: python/PIL/report.py (deflated 21%)
adding: python/PIL/MspImagePlugin.py (deflated 62%)
adding: python/PIL/bcspImagePlugin.py (deflated 51%)
adding: python/PIL/contour.py (deflated 65%)
adding: python/PIL/_imagingcms.py (deflated 78%)
adding: python/PIL/FpxImagePlugin.py (deflated 66%)
adding: python/PIL/fitzImagePlugin.py (deflated 69%)
adding: python/PIL/_typing.py (deflated 54%)
adding: python/PIL/XmImagePlugin.py (deflated 62%)
adding: python/PIL/cifImagePlugin.py (deflated 75%)
adding: python/PIL/cmapImagePlugin.py (deflated 64%)
adding: python/PIL/ImageMode.py (deflated 66%)
adding: python/PIL/_main_.py (deflated 26%)
adding: python/PIL/tifImagePlugin.py (deflated 64%)
adding: python/PIL/_util.py (deflated 46%)
adding: python/PIL/Hdf5StubImagePlugin.py (deflated 58%)
  
```



4. Create the Lambda Layer



Compatible architectures - optional | [Info](#)
Choose the compatible instruction set architectures for your layer.

Compatible runtimes - optional | [Info](#)
Choose up to 15 runtimes.

License - optional | [Info](#)

[Cancel](#) [Create](#)

ARN: arn:aws:lambda:eu-north-1:095074107161:layer:pillow-layer:1

Attach the Pillow layer to your Lambda

AWS | [Search](#) | [\[Alt+S\]](#) | [Europe](#)

Lambda > Add layer

Add layer

Function runtime settings

Runtime: Python 3.12 | Architecture: x86_64

Choose a layer

Layer source | [Info](#)
Choose from layers with a compatible runtime and instruction set architecture or specify the Amazon Resource Name (ARN) of a layer version. You can also [create a new layer](#).

☐ AWS layers
Choose a layer from a list of layers provided by AWS.

☒ Custom layers
Choose a layer from a list of layers created by your AWS account.

☐ Specify an ARN
Specify a layer by providing the ARN.

Custom layers
Layers created by your AWS account that are compatible with your function's runtime.

Version

[Cancel](#) [Add](#)

5. Lambda Code

In the Code tab of `lambda_function`

```
import boto3
from PIL import Image
import os
from urllib.parse import unquote_plus

s3 = boto3.client('s3')

def lambda_handler(event, context):
    try:
        # Extract bucket + object key
        record = event['Records'][0]
        source_bucket = record['s3']['bucket']['name']
```

```
object_key = unquote_plus(record['s3']['object']['key']) # IMPORTANT FIX

print("Processing:", source_bucket, object_key)

# Download file from S3
tmp_path = f"/tmp/{os.path.basename(object_key)}"
s3.download_file(source_bucket, object_key, tmp_path)

# Open and resize image
img = Image.open(tmp_path)
img = img.resize((300, 300))

# Output path
output_bucket = "image-processed-sakshi"
output_key = f"processed-{os.path.basename(object_key)}"
output_path = f"/tmp/{output_key}"

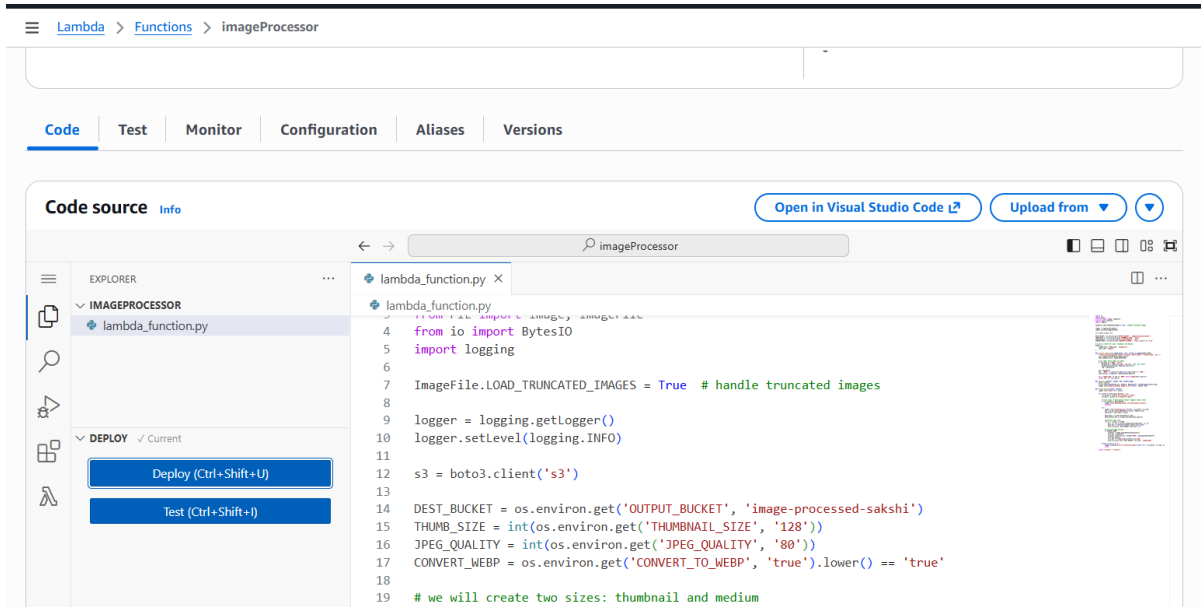
img.save(output_path)

# Upload processed image
s3.upload_file(output_path, output_bucket, output_key)

print("Upload successful:", output_key)

return {"status": "success", "file": output_key}

except Exception as e:
    print("Error:", str(e))
    raise e
```



Click **Deploy**.

6. S3 Trigger Setup (image-source-sakshi → Lambda)

We want: every upload to image-source-sakshi triggers the Lambda.

Lambda → Configuration → Triggers → Add trigger.

The screenshot shows the 'Add trigger' page in the AWS Lambda console. The 'Trigger configuration' section is expanded, showing the following details:

- Provider:** S3 (aws, asynchronous, storage)
- Bucket:** s3/image-source-sakshi (Bucket region: eu-north-1)
- Event types:** All object create events
- Prefix - optional:** e.g. images/
- Suffix - optional:** .png
- Recursive invocation:** If your function writes objects to an S3 bucket, ensure that you are using different S3 buckets for input and output. Writing to the same bucket increases the risk of creating a recursive invocation, which can result in increased Lambda usage and increased costs. [Learn more](#)

The 'Add' button is highlighted in orange at the bottom right.

7. Test the Pipeline End-to-End

Upload a test image

image-source-sakshi:

The screenshot shows the Amazon S3 console's 'Upload' interface. At the top, the breadcrumb navigation indicates the path: Amazon S3 > Buckets > image-source-sakshi > Upload. Below this, a section titled 'Files and folders (2 total, 85.1 KB)' contains a table with one entry: 'Bl.jpeg', which is an 'image/jpeg' file of size '18.3 KB'. To the right of this table are buttons for 'Remove', 'Add files', and 'Add folder'. Below the table, there are expandable sections for 'Destination' (showing the path 's3://image-source-sakshi'), 'Destination details', 'Permissions', and 'Properties'. At the bottom right, there are 'Cancel' and 'Upload' buttons.

Check processed bucket

Look at image-processed-sakshi.

The screenshot displays the 'image-processed-sakshi' bucket page in the Amazon S3 console. The 'Objects' tab is selected, showing a table of four objects: 'processed-Bl.jpeg' (17.3 KB), 'processed-Cloud Computing.png' (50.3 KB), 'processed-Discrete-Mathematics.jpg' (12.0 KB), and 'processed-NLP.jpg' (14.6 KB). All objects are of 'Standard' storage class and were last modified on December 10, 2025. The interface includes various action buttons like 'Copy S3 URI', 'Download', 'Delete', and 'Upload'.

Check logs

The screenshot shows the AWS CloudWatch console. The left sidebar contains navigation links for 'CloudWatch', 'Alarms', 'AI Operations', 'Application Signals (APM)', 'Infrastructure Monitoring', 'Logs', 'Metrics', and 'Network Monitoring'. The main area is titled 'Log management' and shows the configuration for the 'ImageProcessor' lambda function. It includes tabs for 'Log streams', 'Tags', 'Anomaly detection', 'Metric filters', 'Subscription filters', 'Contributor Insights', 'Data protection', and 'Field indexes'. The 'Log streams' tab is active, displaying a table of four log streams with their 'Last event time'. The top of the console shows the AWS account ID '0950-7410-7161' and the user 'Sakshi Jain'.